



HASHDIT

HashDit

Smart Contract Security

ListaDao AtlasOracleAdaptor Audit Report

Security Audit Report

ListaDao

20 May 2026 - 22 May 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	20 May 2026 – 22 May 2026
Overall Risk [Optional]	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-new-contracts/pull/26
Commit	feat/atlas-oracle-wrapper

Smart Contract List

No.	Contract Name	Verdict	Details
1	AtlasOracleAdaptor.sol	Informational	[I01]
			[I02]
			[I03]
			[I04]
			[I05]
			[I06]
			[I07]

Findings

[I01] Contract and file name misspelling of Atlas as Altas

Contract AtlasOracleAdaptor.sol

Severity **Informational**

Description

The contract, source file, and revert strings in AltasOracleAdaptor.sol use the misspelling Altas instead of Atlas, while the product is documented as Atlas Oracle throughout the NatSpec, deploy script filename (deployAtlasOracleAdaptors.sol), and user facing description.

The contract is named AltasOracleAdaptor and the revert strings "AltasOracleAdaptor/zero-feed" and "AltasOracleAdaptor/feed-decimals-not-18" carry the same typo.

Recommendation

Rename the contract and source file to AtlasOracleAdaptor and update the two revert strings to "AtlasOracleAdaptor/zero-feed" and "AtlasOracleAdaptor/feed-decimals-not-18" before deployment so that the on chain name aligns with the documented product name and the deploy script.

Status Fixed in commit [4b2f04c](#), all incorrect typographical errors are now corrected.

[I02] Inconsistent handling of version() and description() metadata

Contract AtlasOracleAdaptor.sol

Severity **Informational**

Description

In AltasOracleAdaptor.sol, the description() function forwards the call to the underlying Atlas feed and returns its description, while the version() function returns a hardcoded value of 1 instead of forwarding to the underlying feed's version. This mixed convention is confusing for integrators and off chain monitoring tools.

Description

Forwarding both values gives more accurate visibility into the underlying feed for off chain monitoring, while hardcoding both values gives more deterministic behavior.

JavaScript

```
function description() external view returns (string memory) {  
    return atlasFeed.description();  
}
```

```
function version() external pure returns (uint256) {
    return 1;
}
```

Recommendation

Choose a single convention for both metadata functions. Either forward both version() and description() to the underlying Atlas feed for consistent off chain monitoring, or hardcode both values for deterministic behavior. Avoid mixing the two approaches.

Status

Fixed in commit [a01bac4](#), the version() function now calls the underlying atlasFeed.version function.

[I03] Price rescaling in _scale can possibly truncate toward zero

Contract

AtlasOracleAdaptor.sol

Severity

Informational

Description

Description

The _scale function in AtlasOracleAdaptor uses FullMath.mulDiv(ans, 1e8, 1e18), which reduces to ans / 1e10 and truncates toward zero. Any token priced under 1e10 (i.e. 1e-8 USD) would round down to zero, effectively preventing the use of such a low value token.

JavaScript

```
/**
 * @dev Rescale an 18-decimal answer to 8 decimals via FullMath.mulDiv
 * to
 * avoid intermediate overflow. Non-positive answers are coerced to
 * zero,
 * which ResilientOracle treats as INVALID_PRICE (preventing a
 * signed-to-
 * unsigned underflow wrap when uint256(answer) is taken downstream).
 */
function _scale(int256 ans) internal pure returns (int256) {
    if (ans <= 0) return 0;
    return int256(FullMath.mulDiv(uint256(ans), TARGET_SCALE,
SOURCE_SCALE));
}
```

Currently, this scenario is unrealistic since none of the 20 listed assets are anywhere near that price range, and even if it did occur, ResilientOracle already guards against a zero price by treating it as INVALID_PRICE. As such, this is noted for completeness for possible future feeds for low valued tokens or during times of extreme token price drop.

Recommendation

No code change is required given the current asset list and the downstream INVALID_PRICE protection in ResilientOracle. For clarity, add a NatSpec note on `_scale` stating that the conversion truncates toward zero and is intended for assets priced well above $1e-10$, so future operators do not list ultra low priced tokens without revisiting the rescaling math.

Alternatively, if the `int256(FullMath.mulDiv(uint256(ans), TARGET_SCALE, SOURCE_SCALE)) <= 0`, we can return zero directly, which will direct price check logic to ResilientOracle.

Status

Fixed in commit [59eebe3](#), adds sufficient documentation indicating warnings for operators for low value tokens.

[I04] Oracle Migration considerations outside of Atlas Oracle

Contract AtlasOracleAdaptor.sol

Severity **Informational**

Description

There are 7 tokens that currently hold a mainOracle that corresponds to a specific BinanceOracle. This delegates full trust to the underlying mainOracle (i.e the bound validator check is not performed between pivot and main oracle within the ResilientOracle to use the fallbackOracle or revert)

- BBTC
- OIK
- EGL1
- Aster
- WLFI
- Take
- CDL

Description

The suggested migrated oracle is the AtlasOracle, however, considerations for more reliable feeds and/or configurations could be explored for two specific tokens where reliable feeds are available, namely, ASTER and WLFI.

Token	Main Oracle (Binance Oracle)	Migrated Oracle (Atlas Oracle)	Possible Oracles
ASTER	ASTER/USD	ASTER/USD	ASTER/USD (Chainlink) ASTER/USD (Pyth)
WLFI	WLFI/USD	WLFI/USD	WLFI/USD (Chainlink) WLFI/USD (Redstone) WLFI/USD (Pyth)

Based on the above configuration we can see that:

- The WLF token can use a main + pivot + fallback oracle (Chainlink + Redstone + Atlas)
- The ASTER token can use a main + pivot oracle + fallback oracle (Chainlink + Pyth + Atlas)

Recommendation

Consider reviewing the migration plan of the above mentioned tokens, although if oracles outside of Atlas are to be integrated, separate OracleAdaptors would need to be implemented to allow integration considering Chainlink/Redstone/Pyth Contracts code logic

Status

Acknowledged, ListaDao has indicated that they will consider other oracles if available in the future.

[I05] Deploy script validates only feed decimals, not feed identity

Contract

AtlasOracleAdaptor.sol

Severity

Informational

Description

Description

The deployment script script/oracle/deployAtlasOracleAdaptors.sol deploys twenty AltasOracleAdaptor instances using a hardcoded list of Atlas push feed addresses. The only on chain validation performed at deploy time is inside the adaptor constructor, which requires `AggregatorV3Interface(_atlasFeed).decimals() == 18`.

The script does not verify feed identity before broadcast, for example by checking `description()` against the expected symbol, confirming the feed id from the ListaDao Assets spreadsheet, or comparing bytecode/owner against known Atlas Oracle deployments. A wrong but 18 decimal aggregator address would deploy successfully and produce a valid looking adaptor that reports incorrect prices if later wired into production oracle configuration.

Related Code

Deploy script passes `pushContract` directly with no pre deploy checks:

JavaScript

```
for (uint256 i = 0; i < feeds.length; i++) { AltasOracleAdaptor adaptor
= new AltasOracleAdaptor(feeds[i].pushContract);
console.log("AltasOracleAdaptor", feeds[i].symbol, "->",
address(adaptor)); }
```

Constructor only validates non zero address and 18 decimals:

JavaScript

```
constructor(address _atlasFeed) { require(_atlasFeed != address(0),  
"AtlasOracleAdaptor/zero-feed"); require(  
AggregatorV3Interface(_atlasFeed).decimals() == 18,  
"AtlasOracleAdaptor/feed-decimals-not-18" ); atlasFeed =  
AggregatorV3Interface(_atlasFeed); }
```

Recommendation

Extend the deploy script (or a companion verification script run before --broadcast) to assert each feed's description(), a sample latestRoundData() sanity check, and that the address matches the official Atlas feed registry / spreadsheet for the stated feed id. Fail the script on mismatch. Record deployed adaptor addresses in a versioned migration manifest (JSON or markdown) keyed by feed id and symbol for governance review.

Status

Acknowledged, ListaDao has indicated that they will perform necessary oracle address validation before deployment.

[I06] Per asset adaptor mapping relies on manual steps without on chain registry

Contract

AtlasOracleAdaptor.sol

Severity

Informational

Description

Description

The deploy script outputs adaptor addresses via console.log only. There is no on chain registry contract and no machine readable artifact committed in the repository that maps asset → adaptor → underlying Atlas feed. Twenty adaptors are deployed for unique feeds, but several Lista assets share a single feed (e.g. BTCB/USD feed 626 for solvBTC, SolvBTC.DLP, pumpBTC, mBTC; USDT/USD feed 649 for USDF). Manual translation from deploy logs to per asset wrapper configuration is error prone: a correct adaptor could be wired to the wrong collateral, or the right asset could remain on an old aggregator, causing incorrect pricing for borrowing, liquidation, and bound validation.

Related Code

Output is console only; no struct or file records asset → adaptor:

JavaScript

```
AtlasOracleAdaptor adaptor = new  
AtlasOracleAdaptor(feeds[i].pushContract);  
console.log("AtlasOracleAdaptor", feeds[i].symbol, "->",  
address(adaptor)); console.log(" underlying Atlas feed:",  
feeds[i].pushContract);
```

Shared feeds are noted in comments but not expanded to per collateral token addresses:

Recommendation	After deployment, generate and review a migration manifest listing every affected collateral token address, its target adaptor address, underlying Atlas feed address, and feed id. Use a scripted post deploy check that calls latestRoundData() on each adaptor and compares against the underlying feed. Require two person review before governance submits wrapper / ResilientOracle transactions. For shared feeds, explicitly enumerate all collateral assets that must point at the same adaptor in the manifest.
Status	Acknowledged, ListaDao has indicated that they will perform necessary documentation updates after deployment.

[I07] Migration requires oracle wrapper aggregator updates, not only ResilientOracle config

Contract AtlasOracleAdaptor.sol

Severity **Informational**

Description

AltasOracleAdaptor implements AggregatorV3Interface only and is intended to sit behind an existing Lista oracle wrapper (e.g. BinanceOracle) that implements OracleInterface.peek() and applies staleness via timeDeltaTolerance.

This PR deploys adaptors but does not include scripts or documentation in repo for updating per asset aggregator mappings inside those wrappers. Audit notes reference ResilientOracle.setTokenConfig / enableOracle, but if operators update ResilientOracle while leaving the wrapper's internal aggregator pointer on the legacy Binance feed, the migration would not take effect and prices would remain on the old source.

Conversely, updating the wrapper without aligning ResilientOracle roles and enable flags can leave assets on stale or unintended oracle paths.

Related Code

Lista OracleInterface (used by ResilientOracle wrappers) requires peek(asset), which AltasOracleAdaptor does not implement:

Description

JavaScript

```
interface OracleInterface { function peek(address asset) external view
returns (uint256); } contract AltasOracleAdaptor is
AggregatorV3Interface { ... }
```

Deploy script only instantiates adaptors; it does not call any wrapper or ResilientOracle setter:

JavaScript

```
for (uint256 i = 0; i < feeds.length; i++) { AltasOracleAdaptor adaptor
= new AltasOracleAdaptor(feeds[i].pushContract);
console.log("AltasOracleAdaptor", feeds[i].symbol, "->",
address(adaptor)); } vm.stopBroadcast();
```

Consider publishing a step by step migration runbook that explicitly lists, per asset and execute changes in a single maintenance window with rollback addresses documented.

Recommendation

1. deployed adaptor address,
2. wrapper contract and function used to set the aggregator/source address
3. Any ResilientOracle setTokenConfig / setOracle / enableOracle changes
4. Post migration verification (peek() parity, decimals(), updatedAt, staleness tolerance).

Status

Acknowledged, ListaDao has indicated that they will wire the adaptor into ResilientOracle directly via setTokenConfig() to perform necessary checks.